

Documentação Técnica

Sistema Multi-Carregador de Extração de Logs (LCTC)

`journal_strip_multi_chargers.py`

Desenvolvido por Brunesi

11 de maio de 2026

Conteúdo

1	Visão Geral do Sistema	2
1.1	Objetivo	2
1.2	Arquitetura de Software	2
2	Estruturas de Dados e Classes	3
2.1	Dataclasses de Estado	3
2.2	Gerenciamento de Logs (<code>EventLogger</code>)	3
2.3	O Hub de Eventos (<code>EventHub</code>)	3
3	Lógica de Processamento e Protocolos	4
3.1	Filtragem de Dados (Regex)	4
3.2	Detecção de Sessões	4
4	Fluxo de Execução e Diagramas	5
4.1	Diagrama de Fluxo Lógico	5
4.2	Controle de Concorrência (Semaforização)	7
5	Interface de Usuário (Textual TUI)	8
5.1	Estrutura da TUI	8
5.2	Exemplo de Saída Visual	8
6	Manual de Métodos Principais	9
6.1	<code>load_chargers(path)</code>	9
6.2	<code>run_remote_journal(...)</code>	9
6.3	<code>save_file(...)</code>	9

Capítulo 1

Visão Geral do Sistema

1.1 Objetivo

O `journal_strip_multi_chargers.py` é uma ferramenta de engenharia avançada projetada para realizar a coleta massiva e análise de logs em infraestruturas de carregamento de veículos elétricos (EV). O foco principal é extrair payloads hexadecimais do serviço `chargepoint.service`, permitindo a reconstrução de sessões de carregamento nos protocolos **CHAdemo** e **CCS**.

1.2 Arquitetura de Software

O script utiliza o paradigma de **Programação Assíncrona** através da biblioteca `asyncio`. Diferente de modelos multi-thread tradicionais que consomem muita memória, este modelo permite gerenciar centenas de conexões SSH simultâneas em um único loop de eventos, aguardando I/O sem bloquear o processamento principal.

Capítulo 2

Estruturas de Dados e Classes

2.1 Dataclasses de Estado

O código utiliza `dataclasses` para garantir imutabilidade e tipagem forte:

- **Charger**: Armazena a identidade do hardware (`custom_id` e `ip`).
- **JournalReadStats**: Contador volátil para monitoramento de linhas brutas vs. linhas filtradas.
- **ChargerEvent**: Objeto de transporte que notifica a interface sobre mudanças de estado (ex: "conectando", "salvando").
- **ProcessResult**: Resumo final da operação de um carregador individual.

2.2 Gerenciamento de Logs (EventLogger)

A classe `EventLogger` é responsável pela persistência do histórico da aplicação.

- **Método `write`**: Adiciona timestamps e formata mensagens.
- **Segurança**: Utiliza `flush()` imediato para garantir que em caso de crash do script, os logs até aquele ponto não sejam perdidos no buffer do SO.

2.3 O Hub de Eventos (EventHub)

Atua como um *Broker* de mensagens interno. Quando uma corrotina de processamento de carregador detecta um progresso, ela emite um `ChargerEvent` para o Hub. O Hub então distribui essa informação para três destinos:

1. O arquivo de log físico (via `EventLogger`).
2. O console padrão (`Stdout`), se o modo silencioso estiver desligado.
3. A fila assíncrona da interface *Textual*, se a TUI estiver ativa.

Capítulo 3

Lógica de Processamento e Protocolos

3.1 Filtragem de Dados (Regex)

O script busca o padrão 04 64 <byte>. Este padrão é específico para as tramas de comunicação serial capturadas no journal do sistema Linux do carregador. A expressão regular `re.compile(r"04 64 (?!\10|\11|\e1)[0-9a-f]{2}.*")` exclui explicitamente bytes de controle não desejados (10, 11, e1).

3.2 Detecção de Sessões

A função `detect_sessions` implementa uma máquina de estados finitos (FSM) simplificada:

Protocolo	Byte Start	Byte End	Ação
CHAdemo	81	ad	Fecha log anterior, inicia .chademo.log
CCS	32	22	Fecha log anterior, inicia .ccs.log

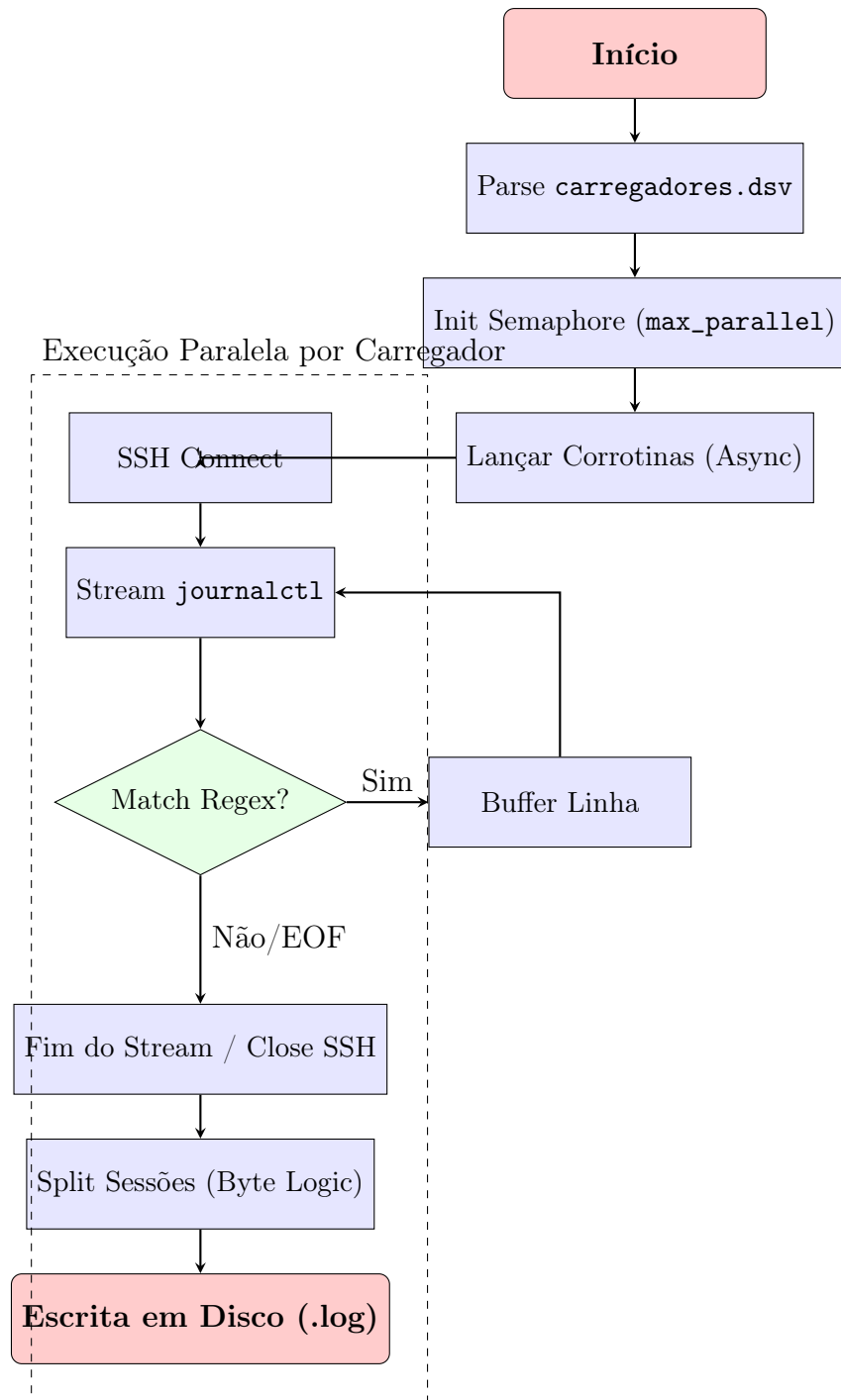
Tabela 3.1: Gatilhos hexadecimais para segmentação de arquivos.

Capítulo 4

Fluxo de Execução e Diagramas

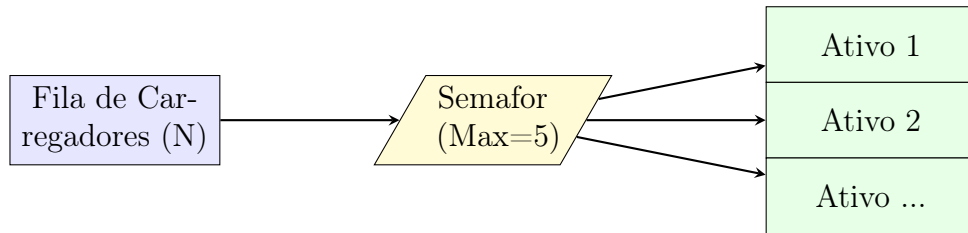
4.1 Diagrama de Fluxo Lógico

O diagrama abaixo detalha o caminho desde a leitura do DSV até a finalização das tarefas.



4.2 Controle de Concorrência (Semaforização)

Diferente de disparar todos os carregadores de uma vez, o uso de `asyncio.Semaphore(max_parallel)` garante que o host não sofra exaustão de descritores de arquivo ou banda de rede.



Capítulo 5

Interface de Usuário (Textual TUI)

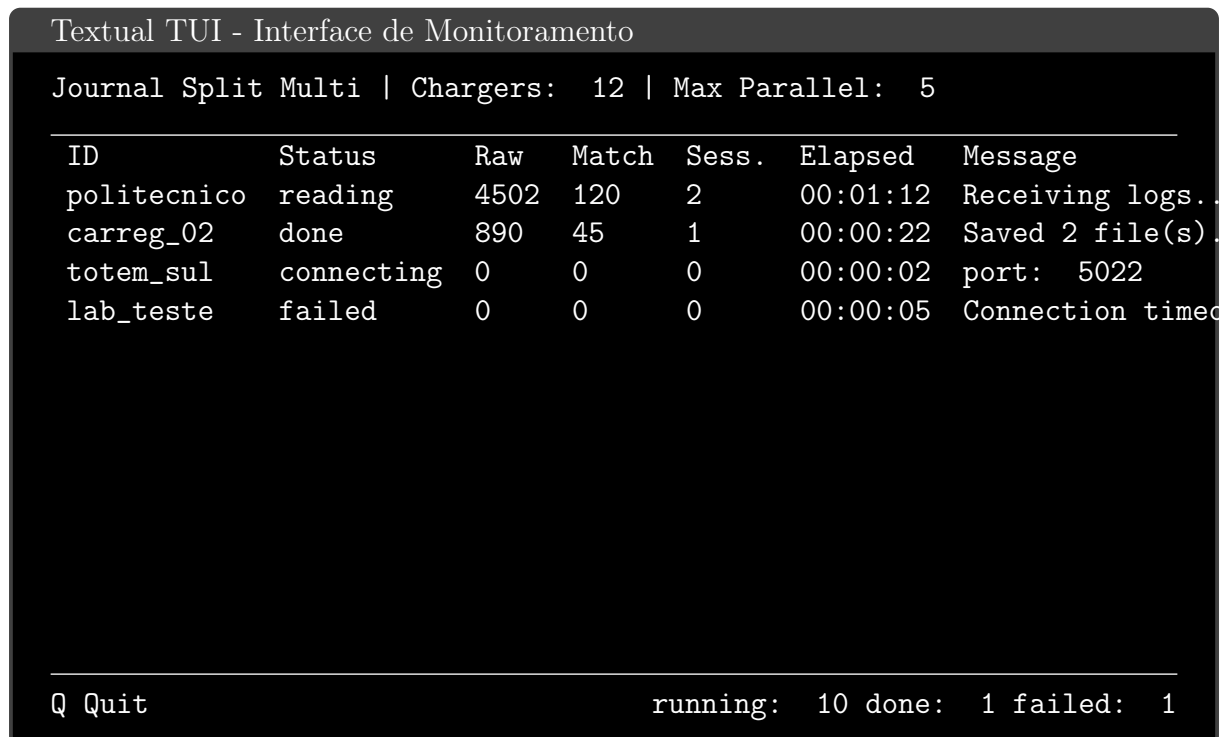
5.1 Estrutura da TUI

O componente `JournalSplitTextualApp` herda de `textual.app.App`. Ele organiza a tela em três áreas:

- **Header:** Título e estatísticas globais.
- **DataTable:** Tabela dinâmica onde cada linha representa um carregador.
- **Footer:** Atalhos de teclado (ex: 'Q' para sair).

5.2 Exemplo de Saída Visual

Abaixo, uma representação gráfica da tabela gerada pelo código em tempo de execução:



The screenshot shows a terminal window titled "Textual TUI - Interface de Monitoramento". The main content area displays a table with 7 columns: ID, Status, Raw, Match, Sess., Elapsed, and Message. The table contains 4 rows of data. Below the table, there is a footer section with a quit prompt "Q Quit" on the left and a summary of the data: "running: 10 done: 1 failed: 1" on the right.

ID	Status	Raw	Match	Sess.	Elapsed	Message
politecnico	reading	4502	120	2	00:01:12	Receiving logs...
carreg_02	done	890	45	1	00:00:22	Saved 2 file(s).
totem_sul	connecting	0	0	0	00:00:02	port: 5022
lab_teste	failed	0	0	0	00:00:05	Connection timed

Q Quit running: 10 done: 1 failed: 1

Capítulo 6

Manual de Métodos Principais

6.1 `load_chargers(path)`

Lê o arquivo DSV. Realiza a limpeza de aspas e espaços. Crucialmente, invoca `validate_unique_custom` para evitar que dois carregadores tentem escrever no mesmo arquivo de saída, o que causaria corrupção de dados.

6.2 `run_remote_journal(...)`

Este é o núcleo da comunicação.

- Abre a conexão SSH.
- Cria um processo remoto executando `journalctl`.
- Usa um gerador assíncrono para ler linha a linha (`async for raw_line in process.stdout`).
- Atualiza estatísticas em tempo real para a TUI através do Hub.

6.3 `save_file(...)`

Garante que os nomes dos arquivos sejam seguros para o sistema de arquivos utilizando a função `safe_filename_part`, que remove caracteres especiais e normaliza acentos para ASCII.